

Module 0 — Foundations toolbox

The book assumes this material but doesn't teach it. We build it from scratch, because every later derivation leans on it. It's a *reference*, not a race — skim what's solid, slow down where it's rusty, and say "skip" on anything you already own cold. Math is written in LaTeX (it typesets in the PDF); in live chat we use plain unicode.

Contents: 0.1 Sums · 0.2 Counting · 0.3 Binomial coefficients · 0.4 Discrete probability & expectation · 0.5 Harmonic numbers · 0.6 Logs, factorials, and Big-O.

0.1 Sums and sigma-notation

$\sum$ ("sigma") just means "add these up."

$$\sum_{k=1}^n a_k = a_1 + a_2 + \cdots + a_n.$$

Here k is the *index* (a dummy name — you can rename it), 1 and n are the lower/upper *bounds*, and a_k is the *summand*. We also write $\sum_{1 \leq k \leq n} a_k$, which means the same thing. An empty sum (no valid k) is 0 by convention.

The three manipulations you'll use constantly

(1) **Pull out a constant factor.** If c doesn't depend on k :

$$\sum_k (c a_k) = c \sum_k a_k.$$

(2) **Split a sum of sums.**

$$\sum_k (a_k + b_k) = \sum_k a_k + \sum_k b_k.$$

(3) **Split or shift the range.**

$$\sum_{k=1}^n a_k = \sum_{k=1}^m a_k + \sum_{k=m+1}^n a_k \quad (\text{split at } m).$$

Reindexing (shifting the index) is the sneaky-useful one. If you let $j = k - 1$, then as k runs $1, \dots, n$, j runs $0, \dots, n - 1$, and

$$\sum_{k=1}^n a_k = \sum_{j=0}^{n-1} a_{j+1}.$$

Renaming or shifting the index changes the *labels*, never the *value*. This is exactly the move that revealed the symmetry in the quicksort sum in Module 1.

Closed forms worth memorizing

- **Constant:** $\sum_{k=1}^n 1 = n$.
- **First n integers:** $\sum_{k=1}^n k = \frac{n(n+1)}{2}$. *Gauss's proof:* write the sum forwards and backwards and add termwise:

$$\begin{array}{r} S = 1 + 2 + \cdots + n \\ S = n + (n-1) + \cdots + 1 \\ \hline 2S = \underbrace{(n+1) + (n+1) + \cdots + (n+1)}_{n \text{ copies}} = n(n+1), \end{array}$$

so $S = \frac{n(n+1)}{2}$.

- **First n squares:** $\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}$.
- **Geometric series** ($r \neq 1$): $\sum_{k=0}^n r^k = \frac{r^{n+1} - 1}{r - 1}$. If $|r| < 1$, letting $n \rightarrow \infty$:

$$\sum_{k=0}^{\infty} r^k = \frac{1}{1-r}. \text{ (You'll use this constantly with generating functions in Module 3.)}$$

Telescoping — the trick behind the quicksort solve

If the summand is a *difference of consecutive terms*, almost everything cancels:

$$\sum_{k=1}^n (b_{k+1} - b_k) = b_{n+1} - b_1.$$

(Every interior term appears once with $+$ and once with $-$.) This is the discrete analog of the fundamental theorem of calculus, and it's precisely why "divide by $N(N+1)$ to make it telescope" worked in Module 1. Whenever you can massage a recurrence into $b_k - b_{k-1} = (\text{something summable})$, you've basically solved it.

0.2 Counting

Two rules generate almost all of basic counting.

Rule of product (AND). If a process is a sequence of independent choices — the first with a ways, the second with b ways, and so on — the number of outcomes is $a \cdot b \cdots$. *Example:* a 4-character password from 26 letters (repeats allowed) is 26^4 .

Rule of sum (OR). If you must pick from one of several disjoint (non-overlapping) groups of sizes $a, b, \dots$, the number of choices is $a + b + \dots$.

Permutations (order matters)

How many ways to arrange n distinct items in a row? First slot: n choices; next: $n - 1$; ...; last: 1. By the product rule,

$$n! = n \cdot (n - 1) \cdot (n - 2) \cdots 2 \cdot 1 \quad (\text{"}n \text{ factorial"}).$$

Convention: $0! = 1$ (there is exactly one way to arrange nothing). So $1! = 1$, $2! = 2$, $3! = 6$, $4! = 24$, $5! = 120$ — factorials explode fast.

Arranging only k of the n (an ordered selection, a " k -permutation"):

$$n \cdot (n - 1) \cdots (n - k + 1) = \frac{n!}{(n - k)!}.$$

Example: ways to pick 1st/2nd/3rd place from 8 runners $= 8 \cdot 7 \cdot 6 = 336 = \frac{8!}{5!}$.

Combinations (order doesn't matter)

How many ways to choose a k -element subset from n items, ignoring order?

Take the ordered count $\frac{n!}{(n-k)!}$ and divide out the $k!$ orderings of each chosen subset:

$$\binom{n}{k} = \text{"}n \text{ choose } k\text{"} = \frac{n!}{k! (n - k)!}.$$

Example: the number of 5-card poker hands from 52 is $\binom{52}{5} = 2,598,960$. Order of dealing doesn't matter, so we divide by $5!$.

The one distinction to keep straight: does rearranging the same items count as different? Yes $\Rightarrow$ permutation (divide by less). No $\Rightarrow$ combination (divide by $k!$ more).

0.3 Binomial coefficients

$\binom{n}{k}$ is called a **binomial coefficient**; it's so central it gets its own identities.

Basic values. $\binom{n}{0} = 1$ (one empty subset), $\binom{n}{n} = 1$, $\binom{n}{1} = n$.

Symmetry. $\binom{n}{k} = \binom{n}{n-k}$. Why: choosing which k to include is the same as choosing which $n - k$ to exclude.

Pascal's rule. $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$. Why (a "combinatorial proof"): fix one particular item x . A k -subset either **contains** x — then choose the other $k - 1$ from the remaining $n - 1$: $\binom{n-1}{k-1}$ ways — or **omits** x — choose all k from the other $n - 1$: $\binom{n-1}{k}$ ways. Disjoint cases, so add them. This rule builds **Pascal's triangle**, each entry the sum of the two above:

$$\begin{array}{ccccccc}
 & & & & 1 & & \\
 & & & 1 & & 1 & \\
 & & 1 & & 2 & & 1 \\
 & 1 & & 3 & & 3 & & 1 \\
 1 & & 4 & & 6 & & 4 & & 1
 \end{array}$$

Binomial theorem (where the name comes from):

$$(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}.$$

Why: expanding $(x + y)(x + y) \cdots (x + y)$ (n factors), a term $x^k y^{n-k}$ arises by choosing x from k of the factors — $\binom{n}{k}$ ways. Two famous consequences (set $x = y = 1$, then $x = 1, y = -1$):

$$\sum_{k=0}^n \binom{n}{k} = 2^n, \quad \sum_{k=0}^n (-1)^k \binom{n}{k} = 0 \quad (n \geq 1).$$

The first counts all subsets of an n -set; the second says there are equally many even- and odd-sized subsets.

Two identities the book uses without comment:

- **Absorption:** $k \binom{n}{k} = n \binom{n-1}{k-1}$. (Handy for sums with a stray factor of k — e.g. computing expectations.)
- **Hockey stick:** $\sum_{i=k}^n \binom{i}{k} = \binom{n+1}{k+1}$. (A diagonal in Pascal's triangle sums to the entry below its end.)

Don't memorize these cold — recognize them, and know they live here when a sum of binomials appears.

0.4 Discrete probability and expectation

This is the machinery for **average-case** analysis, so it matters most.

Setup. A **sample space** Ω is the set of all possible outcomes. An **event** is a subset of Ω . In the **uniform model** (all outcomes equally likely) the probability of an event A is

$$P(A) = \frac{|A|}{|\Omega|} = \frac{\text{favorable outcomes}}{\text{total outcomes}}.$$

So probability, in the uniform case, *is just counting* — which is why §0.2–0.3 came first. General rules: $0 \leq P(A) \leq 1$, $P(\Omega) = 1$, and for disjoint events $P(A \text{ or } B) = P(A) + P(B)$.

Random variable. A number attached to each outcome — a function $X : \Omega \rightarrow \mathbb{R}$. E.g. "number of compares quicksort does on this particular input" is a random variable over the sample space of all input orderings.

Expectation (the average value of X):

$$E[X] = \sum_x x P(X = x) \quad (\text{sum over the values } x \text{ that } X \text{ takes}).$$

Equivalently, average X over every outcome: $E[X] = \sum_{\omega \in \Omega} X(\omega) P(\omega)$. "Average-case cost" means exactly $E[\text{cost}]$.

Linearity of expectation — the single most useful tool

$$E[X + Y] = E[X] + E[Y], \quad E[cX] = c E[X].$$

This holds ALWAYS — even when X and Y are dependent. That's what makes it magical: you can break a complicated total into simple pieces, average each piece separately, and add, without ever worrying about how the pieces interact.

Indicator random variables are how you exploit that. For an event A define

$$\mathbf{1}_A = \begin{cases} 1 & \text{if } A \text{ happens} \\ 0 & \text{otherwise,} \end{cases} \quad \text{then} \quad E[\mathbf{1}_A] = 1 \cdot P(A) + 0 \cdot (1 - P(A)) = P(A).$$

So: to find the *expected number of times* something happens, write that count as a sum of indicators (one per opportunity), and by linearity the expectation is just the **sum of the individual probabilities**. This is *the* recurring technique in the whole book.

Worked example — expected number of fixed points of a random

permutation. Shuffle $1, \dots, n$ uniformly. Let X be the number of positions i with

$\pi(i) = i$. Write $X = \sum_{i=1}^n \mathbf{1}_i$, where $\mathbf{1}_i$ indicates "item i stays put." Each item is equally likely to land anywhere, so $P(\mathbf{1}_i = 1) = \frac{1}{n}$. By linearity,

$$E[X] = \sum_{i=1}^n P(\mathbf{1}_i = 1) = \sum_{i=1}^n \frac{1}{n} = n \cdot \frac{1}{n} = 1.$$

On average exactly one element stays fixed, for every n — and we never had to touch the messy dependence between positions. That's the power of the method.

The models we'll actually use

- **Random permutation model:** all $n!$ orderings of the input equally likely. This is the standard model for analyzing sorting and search-tree algorithms. (It's why quicksort's pivot is equally likely to have any rank, giving the $\frac{1}{N}$ in Module 1.)
- **Random bitstring / random function models** appear later (Chapters 8-9) for strings and hashing. Same expectation toolkit, different sample space.

0.5 Harmonic numbers

Defined as the running sum of reciprocals:

$$H_n = \sum_{k=1}^n \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}.$$

They're the discrete cousin of the integral $\int \frac{dx}{x} = \ln x$, and they appear whenever a " $\frac{1}{k}$ weighting" telescopes out of a recurrence (as in quicksort). The key fact, proved properly in Module 4:

$$H_n \approx \ln n + \gamma, \quad \gamma = 0.5772156 \dots \text{ (Euler's constant).}$$

Why the $\ln n$: $H_n = \sum \frac{1}{k}$ is a rectangle approximation to the area under $\frac{1}{x}$ from 1 to n , which is $\ln n$; the bounded overshoot converges to γ . H_n grows without bound, but agonizingly slowly (H_n first exceeds 10 only around $n \approx 12,367$). Small values: $H_1 = 1$, $H_2 = 1.5$, $H_4 \approx 2.083$, $H_{10} \approx 2.929$, $H_{100} \approx 5.187$ (vs $\ln 100 + \gamma \approx 5.182$ — already spot on).

0.6 Logs, factorials, and Big-O (quick refreshers)

Logarithms. $\log_b x$ is "the power you raise b to, to get x ." Rules:

$$\log(xy) = \log x + \log y, \quad \log \frac{x}{y} = \log x - \log y, \quad \log(x^p) = p \log x.$$

Three bases appear: $\lg = \log_2$ (binary — natural for divide-in-half algorithms), $\ln = \log_e$ (natural — comes out of calculus and integrals), and $\log_{10}$ (rare here).

Change of base: $\log_b x = \frac{\ln x}{\ln b}$, so *all logs differ only by a constant factor*:

$\lg n = \frac{\ln n}{\ln 2} \approx 1.4427 \ln n$. That's why Big-O doesn't care about the base — but the **exact constant does**, which is why Module 1 carefully converted $2 \ln N \approx 1.39 \lg N$.

Stirling's approximation (factorial growth, used from Chapter 4 on):

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n.$$

You rarely need the whole thing early; just know $n!$ grows faster than any exponential.

Big-O and friends — the notation we'll make rigorous in Module 4, stated informally now so it's not a black box:

- $f(n) = O(g(n))$ — f grows *no faster than* g (up to a constant), for large n . Upper bound. Formally: $|f(n)| \leq C g(n)$ for some constant C and all large enough n .
- $f(n) = \Omega(g(n))$ — f grows *at least as fast as* g . Lower bound.
- $f(n) = \Theta(g(n))$ — both: f grows *exactly like* g up to constants. Tight.
- $f(n) \sim g(n)$ ("asymptotic to") — the strongest, and the one this book prefers: $f(n)/g(n) \rightarrow 1$. This keeps the **leading constant**, e.g. $C_N \sim 2N \ln N$. O-notation would only say $C_N = O(N \log N)$ and throw the **2** away.

Rule of thumb for this course: **O tells you the shape; $\sim$ tells you the truth.** We chase $\sim$.

Takeaways

1. **Sums:** pull out constants, split, reindex; know $\sum k = \frac{n(n+1)}{2}$ and the geometric sum; **telescoping** is the engine behind solving recurrences.
2. **Counting:** product & sum rules $\rightarrow$ permutations $n!$ (order matters) vs combinations $\binom{n}{k}$ (order doesn't).
3. **Binomial coefficients:** $\binom{n}{k} = \frac{n!}{k!(n-k)!}$; symmetry, Pascal's rule, binomial theorem, $\sum_k \binom{n}{k} = 2^n$. Recognize absorption & hockey-stick when sums of them appear.
4. **Probability = counting** (uniform model); **expectation** $E[X]$ is average-case cost; **linearity + indicators** turn "expected count" into "sum of probabilities" — the whole book's workhorse.
5. $H_n \approx \ln n + \gamma$ — why "log" shows up in average costs.

6. **Logs differ by a constant** (so Big-O ignores the base, but $\sim$ keeps the constant); we chase $\sim$ (leading term *and* constant), not just O .

→ Do `problem-sets/module-00-problems.md`, then start Module 1. Everything there now has a foundation under it.